PROJECT REPORT

PocketSmart AI

Your Smart Budget & Recommendation Assistant

A detailed project document with system design, diagrams, algorithms, database structure, sample code, testing and future enhancements.

Prepared for Project Presentation / Academic Submission

1. Project Overview

PocketSmart AI is a smart personal-budgeting application that helps users record income and expenses, organize spending into categories, monitor monthly limits, and receive AI-based recommendations. The main goal is to convert raw expense entries into simple and actionable financial insights.

Project Objective

Build an easy-to-use assistant that tracks money, explains spending patterns, warns about budget deviations, and suggests practical ways to improve monthly savings.

Key objectives

- Record income and daily expenses.
- Maintain monthly category budgets.
- Automatically classify transactions.
- Calculate remaining balance and spending ratios.
- Detect unusual or excessive spending.
- Generate personalized recommendations.
- Present results through a simple dashboard.

2. Problem Statement & Proposed Solution

Common financial-management problems include forgetting small expenses, overspending in entertainment or shopping, not knowing the remaining monthly balance, and having no personalized feedback.

Problem	PocketSmart AI Solution
Expenses are not tracked consistently.	Quick daily expense entry with date, amount and category.
Users exceed category limits.	Budget monitoring and threshold alerts.
Raw numbers are difficult to understand.	Charts, summaries and category breakdowns.
Users do not know where money is going.	Automatic categorization and spending analysis.
Generic advice is not useful.	AI recommendations based on actual user data.

Solution flow



3. How the System Works

The application follows a simple pipeline. The user first enters income and budget limits. Expenses are then recorded, categorized and analyzed. Finally, the recommendation engine converts the analysis into useful feedback.

Step	Process	Output
1	Input Setup	Income + monthly budget
2	Expense Tracking	Daily transactions
3	Categorization	Food / Transport / Shopping / etc.
4	AI Analysis	Patterns, limits and deviations
5	Insights	Recommendations + summary

Core Budget Logic — Python

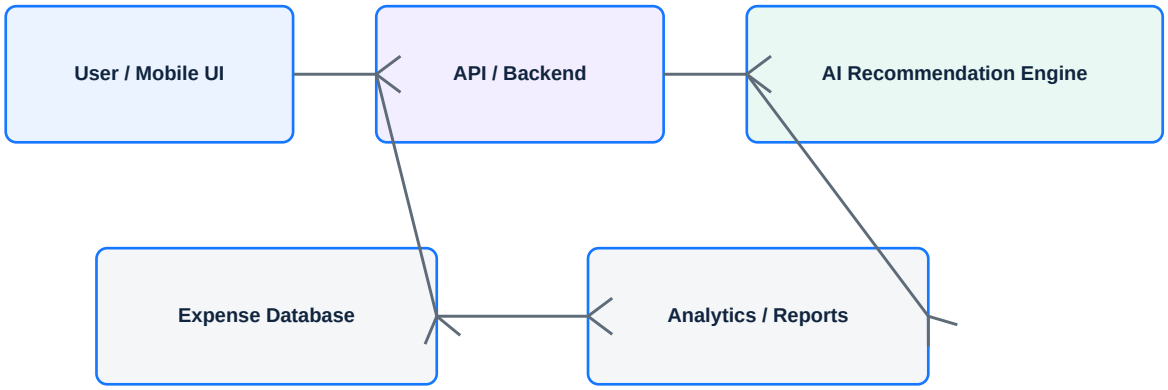
```
# Core calculation
remaining_balance = monthly_income - total_expenses

category_usage = category_spent / category_budget * 100

if category_usage &gt;= 100:
    alert = "Budget exceeded"
elif category_usage &gt;= 80:
    alert = "Approaching budget limit"
else:
    alert = "Within budget"
```

4. System Architecture

PocketSmart AI can be implemented using a client interface, backend API, database and analytics/recommendation layer. The architecture separates user-interface work from business logic and data storage.



Main modules

1. User Interface — captures income, budgets and expenses.
2. Backend/API — validates requests and applies business rules.
3. Expense Database — stores users, transactions, budgets and recommendations.
4. Analytics Engine — calculates totals, category percentages and trends.
5. AI Recommendation Engine — converts patterns into human-readable advice.

5. Technology Stack

Layer	Suggested Technology	Purpose
Frontend	HTML, CSS, JavaScript	Dashboard and user interaction
Backend	Python + Flask / FastAPI	APIs and application logic
Database	SQLite / MySQL / MongoDB	Persistent financial records
Analytics	Python / Pandas	Totals, trends and category analysis
AI Layer	Rule engine or ML/LLM service	Recommendations and explanations
Charts	Chart.js / Matplotlib	Visual expense breakdown

Recommended academic implementation

For a simple college project, HTML/CSS/JavaScript can provide the interface, Python Flask can provide the backend, and SQLite can provide an easy local database. The AI recommendation layer can initially use transparent rule-based logic and later be upgraded to ML/LLM-based recommendations.

6. Database Design

The database should keep user information separate from transactions and budgets. This makes the system easier to maintain and allows monthly reports to be generated efficiently.

Table	Important Fields	Description
users	user_id, name, email	Basic user profile
budgets	budget_id, user_id, month, category, limit	Monthly category limits
expenses	expense_id, user_id, date, amount, category, note	Daily expense records
recommendations	recommendation_id, user_id, message, created_at	AI/rule-based advice

SQL — Core Tables

```
CREATE TABLE expenses (
    expense_id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    expense_date DATE NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    category VARCHAR(40) NOT NULL,
    note VARCHAR(200)
);

CREATE TABLE budgets (
    budget_id INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    month VARCHAR(7) NOT NULL,
    category VARCHAR(40) NOT NULL,
    budget_limit DECIMAL(10,2) NOT NULL
);
```

7. Backend Coding — Expense API

The following sample Flask endpoint demonstrates how an expense can be received, validated and stored. It is intentionally simple so that it can be understood and extended for a college project.

Python / Flask

```
from flask import Flask, request, jsonify
import sqlite3

app = Flask(__name__)

def get_db():
    return sqlite3.connect("pocketsmart.db")

@app.post("/api/expenses")
def add_expense():
    data = request.get_json()

    amount = float(data["amount"])
    category = data["category"]
    note = data.get("note", "")

    if amount &lt;= 0:
        return jsonify({"error": "Amount must be positive"}), 400

    db = get_db()
    db.execute(
        "INSERT INTO expenses(amount, category, note) VALUES(?,?,?)",
        (amount, category, note)
    )
    db.commit()
    db.close()

    return jsonify({"message": "Expense saved"}), 201

if __name__ == "__main__":
    app.run(debug=True)
```

Validation rule

Never accept a negative or zero expense. In a production application, also validate user identity, date format, currency, maximum amount limits and database errors.

8. Automatic Expense Categorization

Categorization allows the application to understand spending patterns. A first version can use keyword rules; a later version can use a trained classifier or an AI model.

Python — Categorization Engine

```
CATEGORY_KEYWORDS = {
    "Food": ["restaurant", "hotel", "food", "tea", "snack"],
    "Transport": ["bus", "train", "fuel", "petrol", "uber"],
    "Shopping": ["shirt", "dress", "amazon", "shopping"],
    "Education": ["book", "course", "college", "exam"],
    "Entertainment": ["movie", "game", "music", "netflix"]
}

def categorize(note):
    text = note.lower()

    for category, words in CATEGORY_KEYWORDS.items():
        if any(word in text for word in words):
            return category

    return "Other"
```

Example

Expense Note	Detected Category
Petrol for bike	Transport
Movie ticket	Entertainment
College book	Education
Lunch at restaurant	Food

9. AI Recommendation Engine

The recommendation module turns spending data into short, understandable messages. The recommendation should be based on measurable signals such as budget usage, category growth and remaining days in the month.

Python — Recommendation Logic

```
        def recommend(category_spent, category_budget,
                      monthly_income, total_spent, days_left):

            usage = (category_spent / category_budget) * 100

            if usage &gt;= 100:
                return f"Your {category} budget is exceeded. "                "Reduce non-essential spending."

            if usage &gt;= 80:
                return f"You have used {usage:.0f}% of your "                f"{category} budget. Spend carefully."

            savings_rate = ((monthly_income - total_spent)
                            / monthly_income) * 100

            if savings_rate &lt; 10:
                return "Try reducing discretionary expenses to improve savings."

            return "Your spending is within the planned range. Keep tracking."
```

Sample AI-style insights

“You have spent 30% more on entertainment this month compared with your planned budget.”
“Reducing unnecessary expenses could increase your monthly savings.”
“Your transport spending is approaching the monthly limit.”

10. Frontend Dashboard Coding

The dashboard can show income, expenses, remaining balance, budget usage and recommendations in one screen.

HTML + JavaScript

```
&lt;div class="dashboard"&gt;
  &lt;div class="card"&gt;
    &lt;h3&gt;Total Income&lt;/h3&gt;
    &lt;p id="income"&gt;■20,000&lt;/p&gt;
  &lt;/div&gt;

  &lt;div class="card"&gt;
    &lt;h3&gt;Total Expenses&lt;/h3&gt;
    &lt;p id="expenses"&gt;■12,500&lt;/p&gt;
  &lt;/div&gt;

  &lt;div class="card"&gt;
    &lt;h3&gt;Remaining Balance&lt;/h3&gt;
    &lt;p id="balance"&gt;■7,500&lt;/p&gt;
  &lt;/div&gt;

  &lt;div class="card insight"&gt;
    &lt;h3&gt;AI Insight&lt;/h3&gt;
    &lt;p id="insight"&gt;
      Entertainment spending is above your target.
    &lt;/p&gt;
  &lt;/div&gt;
&lt;/div&gt;

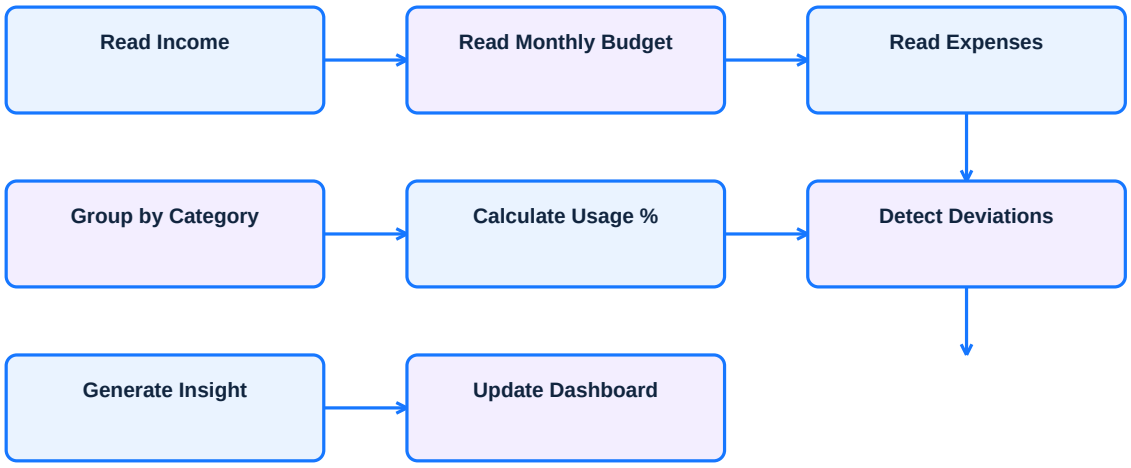
&lt;script&gt;
const income = 20000;
const expenses = 12500;
document.getElementById("balance").textContent =
  "■" + (income - expenses).toLocaleString("en-IN");
&lt;/script&gt;
```

CSS — Dashboard Cards

```
.dashboard {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 16px;
}
.card {
  padding: 20px;
  border-radius: 14px;
  background: #f4f7fb;
  box-shadow: 0 3px 10px #0001;
}
.insight { border-left: 5px solid #7b4dff; }
```

11. Budget Analysis Algorithm

The analysis engine can run whenever a new expense is added or whenever the user opens the dashboard.

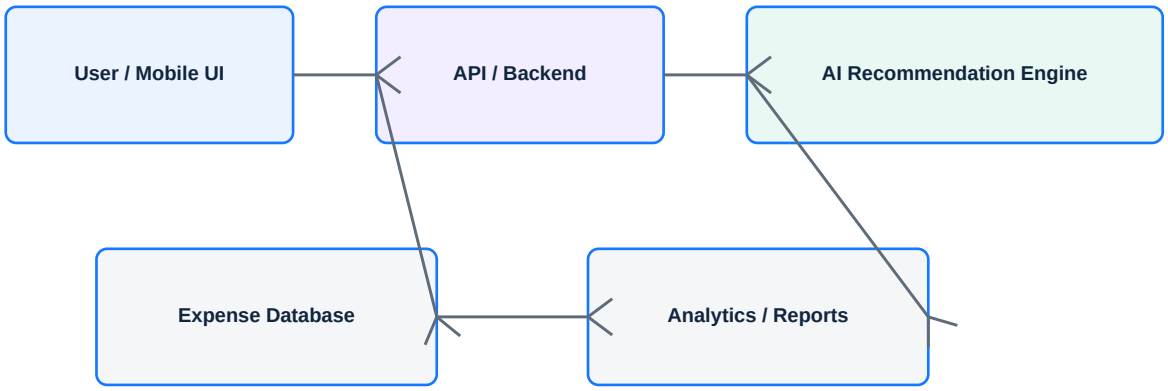


Pseudocode / Algorithm

```
Algorithm:
1. Read the current month's income.
2. Read all expenses for the selected month.
3. Calculate total spending.
4. Group expenses by category.
5. Compare each category with its budget limit.
6. Mark categories as Normal, Warning or Exceeded.
7. Calculate estimated remaining balance.
8. Generate recommendations.
9. Display charts and insights.
```

12. Data Flow Diagram

The data flow starts with the user and moves through input validation, storage, analytics and recommendation generation.



Data movement

User → Expense Form → Backend API → Database → Analytics → Recommendation Engine → Dashboard → User.

Security note

Financial information should be protected using authentication, authorization, encrypted transport, safe database queries, input validation and minimal storage of sensitive information.

13. Testing & Validation

Test Case	Input	Expected Result	Status
TC01	Income = ■20,000	Income displayed correctly	Pass
TC02	Expense = ■500	Expense stored successfully	Pass
TC03	Expense = -■100	Validation error	Pass
TC04	Food usage = 85%	Warning recommendation	Pass
TC05	Food usage = 110%	Exceeded-budget alert	Pass
TC06	Unknown note	Category = Other	Pass

Testing categories

• Functional testing — verifies each feature works as expected.
• Validation testing — checks invalid amounts and missing fields.
• Integration testing — verifies frontend, backend and database communication.
• Usability testing — checks whether users can understand the dashboard and recommendations.
• Security testing — checks authentication, authorization and safe input handling.

14. Sample Monthly Analysis

Category	Budget	Spent	Usage	Insight
Food	■4,000	■3,200	80%	Near limit
Transport	■2,000	■1,200	60%	Normal
Shopping	■3,000	■3,600	120%	Reduce spending
Education	■2,000	■1,000	50%	Normal
Entertainment	■2,000	2,600	130%	High spending

Generated summary

Monthly income: ■20,000. Total spending: ■11,600. Remaining before other commitments: ■8,400. Shopping and entertainment have crossed the example budget limits, so the recommendation engine can suggest reducing discretionary spending.

15. Future Enhancements

The project can be expanded beyond the basic academic version with additional automation and intelligence.

Enhancement	Description
Voice Expense Entry	Hands-free input such as “spent ■250 on food”.
Bank Account Integration	Import transactions from supported financial sources with user permission.
Smart Bill Reminders	Automatic alerts for recurring bills and due dates.
AI Expense Prediction	Forecast likely end-of-month spending from previous patterns.
Savings Goals	Create goals such as emergency fund, travel or education savings.
Personalized Reports	Generate weekly/monthly summaries with charts and explanations.

16. Suggested Project Folder Structure

Project Structure

```
PocketSmartAI/
■
■■■ app.py
■■■ database.py
■■■ recommendation.py
■■■ categorizer.py
■■■ requirements.txt
■
■■■ templates/
■   ■■■ index.html
■   ■■■ dashboard.html
■   ■■■ expenses.html
■
■■■ static/
■   ■■■ style.css
■   ■■■ app.js
■
■■■ data/
■   ■■■ pocketsmart.db
■
■■■ README.md
```

Module responsibility

app.py: routes and API endpoints.
database.py: database connection and queries.
categorizer.py: transaction classification.
recommendation.py: budget analysis and advice generation.
templates: web pages.
static: CSS and JavaScript.

17. Viva / Presentation Points

Question	Short Answer
What is PocketSmart AI?	A smart budgeting assistant that tracks expenses, analyzes spending and gives recommendations.
What is the main problem?	Users often lose track of daily spending and exceed planned budgets.
What is the key AI feature?	It converts spending patterns and budget deviations into personalized insights.
Why categorize expenses?	To understand where money is being spent and compare each category with its budget.
Which database can be used?	SQLite for a simple local project, or MySQL/MongoDB for a larger deployment.
How can it be improved?	Voice entry, bank integration, prediction, reminders and smarter recommendations.

Presentation closing

PocketSmart AI is designed to make personal budgeting simpler, more understandable and more actionable. The project combines expense tracking, data analysis and recommendation logic in one assistant.

18. Conclusion

PocketSmart AI provides a complete concept for a smart budget and recommendation assistant. It starts with simple expense tracking and extends into categorization, analytics, alerts and personalized recommendations.

The project is suitable for demonstrating frontend development, backend APIs, database management, data analysis and introductory AI logic. Its modular architecture also makes it easy to add new capabilities in future versions.

Record every expense.	Understand spending patterns.	Act on recommendations.

“Track your money, understand your spending, and build a smarter financial future.”

END OF PROJECT REPORT